

Analysis of “An Efficient Schema Transformation Technique for Data Migration from Relational to Column-Oriented Databases”

Paper Overview

Problem Addressed

The paper addresses the lack of standard guidelines for migrating data from **Relational Database Management Systems (RDBMS)** to **NoSQL column-oriented databases**. Existing transformation techniques often result in **redundant tables** and **excessive column families**, which lead to inefficient storage usage and degraded query performance.

Motivation

As organizations face the challenges of **Big Data** (volume, velocity, and variety), traditional relational databases often struggle with scalability and flexible schema requirements. While NoSQL databases like HBase offer solutions, the migration process is complex. Current methods that rely on simple denormalization often create unnecessary data duplication, increasing both the storage footprint and the time required to process queries.

Main Contributions

1. **Proposed Technique:** A novel schema transformation method combining **denormalization**, **data access patterns**, and **multiple-nested schemas**.
2. **Efficiency:** Reduction in the number of column families and redundant data compared to baseline methods.

3. **Validation:** Empirical evidence showing significant improvements in **query processing time** (average 29.28% reduction) and **storage space** (average 13.83% reduction) using the DELL DVD and Employees datasets.
-

Data Migration Process

General Process

The migration typically follows the **ETL (Extract, Transform, Load)** framework:

- **Extract:** Retrieving data and schema metadata from the source relational database (e.g., MySQL).
- **Transform:** Converting the relational schema (tables, keys, relationships) into a format suitable for column-oriented storage (e.g., HBase).
- **Load:** Injecting the transformed data into the target NoSQL system using tools like Apache Sqoop.

Key Challenges

- **Schema Mismatch:** Relational databases use normalized, rigid schemas, while column-oriented databases favor denormalized, flexible structures.
 - **Lack of Joins:** NoSQL databases generally do not support complex join operations, requiring data to be pre-aggregated.
 - **Data Redundancy:** Improper denormalization can lead to “data bloat,” where the same information is stored multiple times across different column families.
 - **Rowkey Design:** Selecting an efficient rowkey is critical for performance, as it is the primary way to access data in column-oriented systems.
-

Proposed Approach (In Depth)

Schema Transformation Technique

The authors propose a three-step technique designed to optimize the target schema based on how the data is actually used.

Step-by-step Explanation

1. **Step 1: Denormalization with Read Pattern:** Instead of blindly merging tables, the system analyzes **SQL query logs**. It identifies “access keys” from the `WHERE` clauses of `SELECT` statements to determine which tables are frequently accessed together.
2. **Step 2: Nested and Multiple-Nested Table Merging:**
 - **Nested:** Merging two related tables into one.
 - **Multiple-Nested:** Merging three or more tables into a single column-oriented table based on shared access patterns. This reduces the need for application-level joins.
3. **Step 3: Rowkey Design:** A systematic approach to creating rowkeys that ensure unique identification and efficient retrieval.

Role of Denormalization and Query Patterns

- **Denormalization:** Used to eliminate joins by grouping related data into a single table.
 - **Query Patterns:** Act as a filter for denormalization. Tables are only merged if they share a common access pattern (read pattern), ensuring that the resulting schema is optimized for the specific workload of the application.
-

Proposed Migration Rules

Step 1: Read Pattern Rules

These rules identify the **access key** from SQL `WHERE` conditions:

- **Rule 1:** If the condition is a simple `LIMIT`, the access key is the table's **Primary Key**.
- **Rule 2:** For comparisons between two attributes ($A_i \text{ OPR } A_j$), the access key is either A_i or A_j .
- **Rule 3:** For comparisons with a constant ($A_i \text{ OPR } ?$), the access key is A_i .
- **Rule 4:** For complex Boolean conditions, rules 1-3 are applied recursively to each sub-condition.
- **Rule 5:** If an access key contains sensitive security info, it is dropped or replaced by the Primary Key.

Step 2: Merging Rules

- **Rule 1:** If two queries (Q_i, Q_j) on different tables (P, S) share the same access key, merge P and S into one column family.
- **Rule 2:** If a single query accesses multiple tables using the same access key, merge those tables.
- **Rule 3:** For queries involving a list of tables, merge all of them into a single "Multiple Nested" column family.

Step 3: Rowkey Design Rules

- **Rule 3.1:** If the access key is the RDBMS Primary Key, use it as the Rowkey. Otherwise, combine the access key with the Primary Key to ensure uniqueness.
 - **Rule 3.2:** Ensure all tables from the source are accounted for; if a table wasn't merged, create a new NoSQL table for it using its original Primary Key.
 - **Rule 3.3:** Use short, meaningful names for column families and qualifiers to save storage space (HBase stores these names with every cell).
-

Results and Evaluation

Experimental Setup

- **Source:** MySQL on Ubuntu.

- **Target:** HBase.
- **Tools:** Apache Sqoop for migration, Java connector for application logic.
- **Datasets:** DELL DVD dataset and Employees dataset.

Metrics Used

1. **Database Size (GB):** Total storage occupied by the transformed data.
2. **Response Time (s):** Time taken to execute a set of standard queries (QH1 to QH9).
3. **Storage/Performance Difference (%):** Comparison against a benchmark technique (specifically the work by Zhao et al. [26]).

Key Findings and Conclusions

- **Storage Efficiency:** The proposed method reduced the number of column families (e.g., from 23 down to 15 in one test). This led to an average storage saving of **13.83%**.
- **Performance Gains:** Query processing times improved significantly, with reductions ranging from **12% to 75%** depending on the query type, averaging **29.28%**.
- **Conclusion:** By integrating query access patterns into the denormalization process and using multiple-nested schemas, the technique successfully minimizes data redundancy and optimizes retrieval speed for column-oriented NoSQL databases.